

SoftmaxPIM: An HBM-Based PIM Architecture for Accelerating GeMV–Softmax Execution Pipeline

Taewoon Kang[✉], Namhun Kim[✉], Jihun Lee[✉], Murali Annavaram[✉], *Fellow, IEEE*,
and Gunjae Koo[✉], *Member, IEEE*

Abstract—LLM inference is dominated by GeMV and softmax computations during attention. These computations impose substantial compute and memory bandwidth demands. Prior works have demonstrated that HBM-based PIM architectures effectively accelerate GeMV. However, softmax remains a critical bottleneck due to frequent data movement. In this work, we make the case for computing softmax operations in memory and integrate them with in-memory GeMV operations to provide a comprehensive in-memory LLM accelerator engine. We propose SoftmaxPIM, an in-HBM compute architecture that tightly integrates GeMV with a lightweight near-bank softmax unit. The proposed softmax unit employs an approximation-based softmax formulation that removes reduction operations, enabling the entire GeMV–softmax pipeline to execute within the HBM memory stack. By eliminating host-memory transfers, the proposed design significantly reduces softmax latency. Our simulation results demonstrate up to $42.7\times$ speedup over a CPU and $2.64\times$ over an RTX3090 GPU, as well as up to $1.45\times$ improvement over the prior in-memory attention accelerator AttAcc.

Index Terms—Processing-in-memory, softmax, GeMV, LLM.

I. INTRODUCTION

LARGE language models (LLMs) impose significant computational and memory demands due to their massive parameter sizes and long context lengths. During autoregressive decoding, each token requires a general matrix-vector multiplication (GeMV) between a query vector $\mathbf{q}$ and a key matrix $\mathbf{K}^T$, followed by non-linear operations such as softmax. While prior processing-in-memory (PIM) architectures accelerate GeMV to reduce data movement [1], the intermediate results must still be transferred to GPUs/NPUs for softmax, incurring substantial overhead.

Softmax is a non-linear kernel used to normalize attention scores, as shown in (1):

$$y_k = \frac{\exp(a_k - a_{\max})}{\sum_{i=1}^n \exp(a_i - a_{\max})} \quad (1)$$

This computation involves reduction operations (maximum and summation), which require synchronization and limit parallel efficiency. Prior work reports that softmax accounts for up to 20% of attention latency on parallel processors [2].

Received 18 June 2026; revised 14 July 2026; accepted 18 July 2026. Date of publication 23 July 2026; date of current version 6 August 2026. This work was supported in part by the Institute of Information & Communications Technology Planning & Evaluation (IITP) funded by the Korea Government (MSIT) under Grant RS-2025-02304483 and in part by Chiplet-Based Hub SoC Development Optimized for On-Device AI through ICT Creative Consilience Program under Grant IITP-2026-RS-2020-II201819. (Corresponding author: Gunjae Koo.)

Taewoon Kang, Namhun Kim, Jihun Lee, and Gunjae Koo are with the Department of Computer Science and Engineering, Korea University, Seoul 02841, South Korea (e-mail: taewoon_kang@korea.ac.kr; skagns3571@korea.ac.kr; jihunlee@korea.ac.kr; gunjaekoo@korea.ac.kr).

Murali Annavaram is with the Ming-Hsieh Department of Electrical and Computer Engineering, University of Southern California, Los Angeles, CA 90089 USA (e-mail: annavara@usc.edu).

Digital Object Identifier 10.1109/LCA.2026.3716671

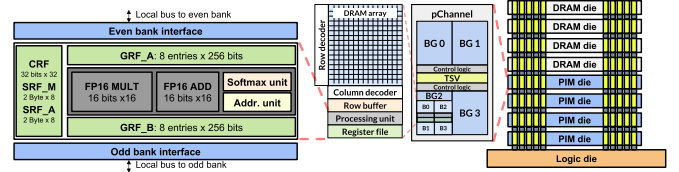


Fig. 1. Architecture of SoftmaxPIM based on HBM-PIM.

Prior PIM-based systems enabled GeMV acceleration by performing matrix operations within HBM [1]. However, the benefits of GeMV acceleration in PIM are curtailed by the need to perform the softmax operation on GPUs/NPUs, which entails moving a significant amount of data from HBM. To address this limitation, we propose SoftmaxPIM, an HBM-based PIM architecture that enables near-bank softmax execution. SoftmaxPIM eliminates synchronization-intensive reductions using an approximate formulation and supports a fully in-memory GeMV–softmax pipeline, significantly reducing both execution time and data movement overhead.

II. PROPOSED ARCHITECTURE

A. Baseline HBM-PIM Architecture

To enable broader adoption of our proposed design, we aim to implement SoftmaxPIM on the commercially available HBM-PIM product from a DRAM vendor [1]. HBM-PIM supports in-memory processing using near-bank SIMD units associated with a pair of banks (i.e., odd and even banks) as shown in Fig. 1. HBM-PIM accelerates GeMV computations (i.e., $\mathbf{q} \times \mathbf{K}^T$ in attention mechanisms) by leveraging these SIMD units through extended DRAM commands. To support bank-level parallel in-memory computations, HBM-PIM operates with an all-bank PIM (AB-PIM) mode, where memory commands are synchronously broadcast to all banks within a pseudo-channel (pCH). Hence, HBM-PIM enables concurrent access to each bank and near-bank computation with minimal DRAM command issues. HBM-PIM can compute GeMV operations efficiently since the near-bank SIMD units concurrently process the multiple data elements in a DRAM row buffer across all banks without transferring the data to host processors.

To support SIMD operations near banks, HBM-PIM incorporates several types of register files. The command register file (CRF) stores operands and control information required for PIM instructions, while the general register file (GRF) holds intermediate and final accumulation results generated during GeMV computation. The scalar register file (SRF) broadcasts scalar operands across SIMD lanes, facilitating efficient vector–scalar operations. All arithmetic operations are executed synchronously, and intermediate results remain within the memory

TABLE I
NEW PIM INSTRUCTIONS FOR SOFTMAXPIM

(a) Added instructions

Op. type	Source A	Source B	Destination
AGEN	GRF	-	-
LDL	Row buffer	-	LUT buffer
MERGE	-	-	-
SCALE	-	-	GRF

(b) Instruction format

	31	30	29	28	27	26	25	24	23	22	21	...	8	7	6	5	4	3	2	1	0
BACC	OPCODE			Unused			SRC0			Unused			SRC0 Idx			Unused					
BMOV	OPCODE			Unused																	

stack (i.e., the associated banks and/or registers) throughout in-memory computations. To initiate PIM instructions on the near-bank SIMD units, HBM-PIM exploits conventional DRAM commands.

B. Near-Bank Softmax Unit

SoftmaxPIM integrates a lightweight softmax unit and an address generation unit alongside the existing SIMD unit, as shown in Fig. 1. The near-bank softmax unit allows SoftmaxPIM to perform softmax using the results of GeMV operations stored in the GRF once the GeMV execution completes. To support in-memory softmax, SoftmaxPIM employs the constant softmax (ConSmax) scheme [3].

$$\text{ConSmax}(x_i) = \frac{\exp(x_i - \beta)}{\gamma} \quad (2)$$

ConSmax approximates the Softmax function as shown in (2). In the equation, β and γ are learnable ConSmax parameters whose values can differ across self-attention heads. Note that these values remain fixed during inference regardless of input tokens [3]. Here, β approximates the maximum value and γ approximates the summation of exponentials, eliminating the global reduction operations required in conventional softmax. Unlike the original ConSmax, SoftmaxPIM adopts a range-limited fixed-point representation to enable more accurate LUT-based exponential computation while preserving efficient bank-level execution.

For LUT-based softmax, SoftmaxPIM introduces four dedicated PIM instructions, as summarized in Table I. The AGEN instruction generates LUT access addresses from the transformed exponent input. LDL fetches the corresponding lookup values from DRAM rows. The MERGE instruction reconstructs the exponential result by combining the integer and fractional components, and SCALE applies the normalization factor $\frac{1}{\gamma}$ before writing the result back to the GRF. To minimize hardware overhead, MERGE and SCALE reuse the existing multiplier used for GeMV computation.

To efficiently support the softmax operation within the PIM unit, we redesign the computation flow to avoid direct evaluation of the exponential function e^x , which is costly in hardware due to its nonlinear nature and lack of native support. Let s_i denote the GeMV output score and define $z_i = s_i - \beta$, where β is a learned normalization parameter. Unlike conventional softmax that subtracts the exact per-vector maximum, ConSmax uses a learned constant to remove the max-reduction step as shown in (2). Therefore, z_i is expected to be concentrated within a limited range after training, although it is not strictly guaranteed to be non-positive for every input. Since values far below zero contribute negligibly due to exponential decay, SoftmaxPIM adopts

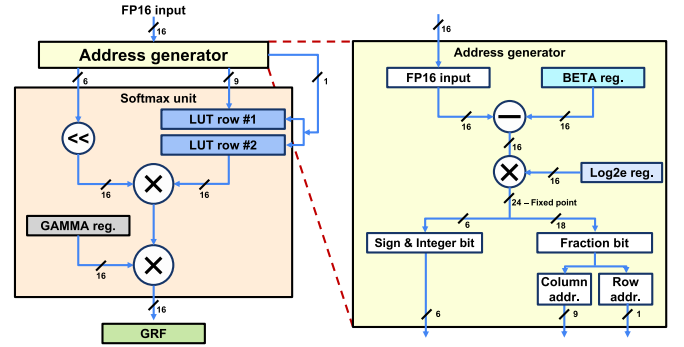


Fig. 2. Near-bank softmax unit.

a range-limited fixed-point representation for the exponent input rather than preserving the full FP16 dynamic range. Values outside the supported fixed-point range are saturated to the nearest representable LUT entry. To leverage this property, we adopt a range-limited fixed-point representation using 18 fractional bits, thus the quantization error is bounded by $|e| \leq 2^{-19}$. This results in a relative error of at most 1.9×10^{-6} in the exponential output, which is significantly smaller than the intrinsic FP16 precision. Before LUT address generation, the eight least-significant bits of the 18-bit fractional field are truncated, forming a 10-bit LUT index.

Building upon this representation, we transform the exponential computation using the following identity equation.

$$e^x = 2^{x \log_2 e}. \quad (3)$$

The input x is first multiplied by a preloaded constant $\log_2 e$ stored in the dedicated register (Log2e reg. in Fig. 2), producing a fixed-point value $y = x \log_2 e$. This value is then decomposed into integer and fractional components, i.e., $y = I + F$, enabling the exponential to be expressed as

$$e^x = 2^I \cdot 2^F \quad (4)$$

As shown in Fig. 2, the integer component I is directly mapped to shift operations, while the fractional component F is used to generate fine-grained memory addresses for LUT-based approximation. Specifically, the fractional part is partitioned into two segments: the upper bits are used as DRAM column addresses, and the lower one bit selects the DRAM row to activate the dedicated LUT rows. The dedicated address generator computes the LUT row and column addresses from the fixed-point representation of y , and the LDL instruction loads the selected 2^F entries from reserved DRAM rows into the local LUT buffer. The retrieved LUT value approximates 2^F , then this value is combined with the shift-based result of 2^I through a multiplication operation. Finally, the result is scaled by the normalization factor $\frac{1}{\gamma}$ using the SCALE instruction to complete the softmax.

III. EXECUTION FLOW

A. GeMV Operation

When the host launches a GeMV kernel, it first performs a preparation phase in which the GeMV command sequence is written into the CRF and the input vector elements are placed into the GRF. The input matrix is tiled and stored in DRAM rows reserved for PIM executions. After initializing the PIM registers, the host transitions the HBM stack into all-bank (AB) mode. In

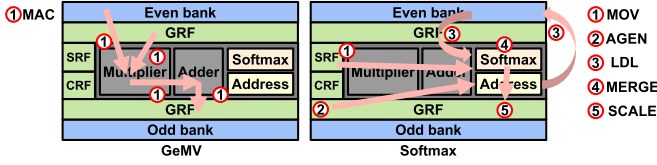


Fig. 3. Execution flows of GeMV and softmax.

AB mode, a single-column command ignores the bank address and bank group fields to fetch the same 256-bit slice from all 16 banks simultaneously. After opening the key row, the HBM is set to all-bank-PIM (AB-PIM) mode to initiate in-memory computation. In AB-PIM mode, each column command triggers the execution of the corresponding PIM instructions stored in the CRF while simultaneously delivering a 4 KB burst (32 B per bank with 128 processing units) to the local datapath. The program counter in the CRF is automatically advanced, enabling lock-step execution across all banks without driving the external I/O interface.

For an $N \times N$ matrix, a single GeMV operation requires $\frac{n}{16} \times \frac{n}{128} \times$ column commands. Each command performs vector-vector multiplication using the 16-lane FP16 SIMD unit associated with odd/even banks, then the generated partial results are accumulated locally. The execution flow of the GeMV computation (i.e. denoted as the MAC operation) is depicted on the left side of Fig. 3.

B. Softmax Operation

The right side of Fig. 3 describes the execution flow of the softmax operation. The numbers denote the PIM instructions for softmax. Note that SoftmaxPIM executes the softmax operation using the operands in the GRF resulting from the GeMV operations, thus the softmax operation does not require data movements from the host processor. For each GeMV-softmax invocation, the host pre-loads the corresponding β and γ into the SRF, and the same values are applied to all elements of the resulting $N \times 1$ vector. The parameters are loaded before the GeMV execution to minimize the configuration overhead caused by the HBM-PIM mode transitions. Once the GeMV operation completes, these parameters are transferred from the SRF into the softmax unit's local configuration registers *BETA* and *GAMMA* using MOV (step ①).

The softmax execution follows steps ②–⑤ in Fig. 3, where the softmax unit performs LUT-based exponential computation using reserved DRAM rows in each bank. For each FP16 input in the GRF, the AGEN instruction invokes the address generator to compute LUT addresses from the scaled input $x_i = S_i - \beta$. The input is first multiplied by the preloaded $\log_2 e$ constant to transform the exponential into base-2 form, $y = x_i \log_2 e$. The resulting fixed-point value is then decomposed into integer and fractional components.

The integer component drives shift-based scaling, while the fractional component generates fine-grained LUT addresses. The fractional bits are mapped to DRAM row and column addresses to access precomputed 2^F LUT values. The LUT is organized across two DRAM rows to improve access flexibility and layout efficiency. Due to the DRAM column access constraints, accesses to different LUT entries need to be serialized. Accordingly, two consecutive LDL instructions are issued to fetch the required LUT values from the row buffers of the

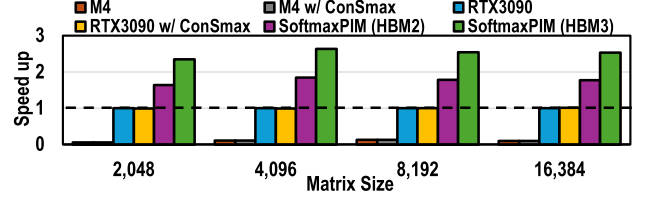


Fig. 4. Normalized speedup by SoftmaxPIM.

corresponding rows, as configured in the CRF. Since each 128-element group follows the same fixed sequence with two LDL instructions, the serialized LUT access latency is deterministic and does not introduce input-dependent pipeline imbalance. After retrieving the fractional component, the MERGE instruction computes the exponential result by multiplying the shift-based integer component with the LUT-based fractional component (i.e., $e^x = 2^I \cdot 2^F$) using the existing multiplier. Finally, SCALE multiplies the result with the normalization factor $\frac{1}{\gamma}$ and writes the final softmax output back to the GRF.

For an $N \times 1$ output vector, SoftmaxPIM processes 128 FP16 elements per PIM instruction, matching the near-bank datapath width of the baseline HBM-PIM. For each 128-element group, SoftmaxPIM first executes one MOV instruction to transfer the parameters (i.e. γ and β) to the softmax unit. It then executes one AGEN to generate LUT addresses, two LDL instructions to fetch the selected 2^F values, one MERGE to reconstruct the exponential value, and one SCALE for normalization. Thus, the complete softmax computation requires $\frac{N}{128} \times 6$ PIM commands.

IV. EVALUATION

A. Evaluation Methodology

To evaluate the proposed SoftmaxPIM architecture, we use a DRAMSim3-based, cycle-accurate in-house simulator [4]. SoftmaxPIM executes a fused kernel that computes an $(N \times N) \times (N \times 1)$ GeMV followed by softmax operation to generate an $N \times 1$ result vector. We vary N from 2,048 to 16,384, in powers of two, to study the scalability of SoftmaxPIM. This range covers the model dimensions of representative open-weight LLMs, such as Llama 3 8B, 70B, and 405B models, which use model dimensions of 4,096, 8,192, and 16,384, respectively [5]. SoftmaxPIM can support larger or rectangular weight matrices through tiling without modifying the proposed datapath. We compare the performance of SoftmaxPIM against FP16-capable processors, including Apple M4 CPU and NVIDIA RTX 3090 GPU. The peak FP16 throughput of M4 CPU and RTX 3090 GPU is approximately 1.5 TFLOPS and 35.6 TFLOPS, respectively [6], [7]. For the HBM2 evaluation, we employ the HBM-PIM configuration [1] derived from the JEDEC specification [8]. For HBM3, we apply the parameters presented in AttAcc [9]. In both settings, four of the eight DRAM dies in a single HBM stack are configured as PIM dies. We estimate the area and power consumption of SoftmaxPIM by synthesizing the hardware logic with Synopsys Design Compiler using a 14 nm SAED FinFET library. The synthesized results are then scaled to 130 nm using DeepScaleTool [10], assuming a $10 \times$ DRAM-process area scaling factor [11].

B. Performance

Fig. 4 exhibits the speedup by SoftmaxPIM across various matrix dimensions, normalized to the RTX 3090 GPU.

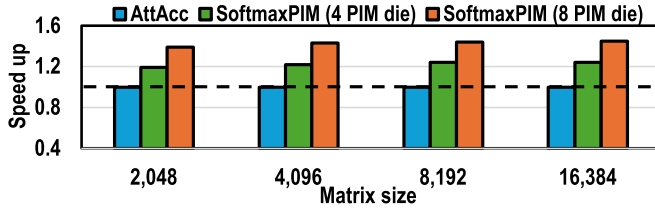


Fig. 5. Performance comparison of GeMV-softmax execution.

TABLE II
CONVERSION LOSS AND PERPLEXITY COMPARISON

Method	Exp. MAE	Softmax MAE	PPL	Token Acc.
Softmax baseline	—	—	43.67	35.97%
ConSmax	1.88×10^{-2}	1.97×10^{-4}	46.70	35.01%
SoftmaxPIM	6.29×10^{-5}	3.66×10^{-7}	46.72	35.04%

SoftmaxPIM achieves $1.64\text{--}1.85\times$ speedup with HBM2 and $2.35\text{--}2.64\times$ speedup with HBM3 over the RTX 3090 GPU, and up to $42.7\times$ over the M4 CPU. The evaluation results in the figure include the performance of the baseline systems with ConSmax (i.e. M4 w/ ConSmax and RTX3090 w/ ConSmax) to reveal the performance gains by ConSmax. These results indicate that ConSmax alone improves the performance of CPU and GPU systems marginally since the performance of GeMV-softmax compute pipeline is dominated by GeMV computation, data movement, and per-element exponential/scaling operations. In contrast, SoftmaxPIM eliminates post-GeMV data transfers and maximizes parallel computations of GeMV-softmax pipeline inside an HBM stack, thus SoftmaxPIM can achieve significant performance gains.

Fig. 5 compares the GeMV-softmax micro-kernel execution time under the HBM3 configuration with the prior PIM-based solution AttAcc [9]. The performance of SoftmaxPIM is normalized to AttAcc. Since AttAcc uses 8 PIM dies by default, we use it as the baseline and evaluate SoftmaxPIM with 4- and 8-PIM-die configurations. Unlike AttAcc that partitions matrix rows across multiple banks and executes softmax on the logic die after accumulation, SoftmaxPIM maps each matrix row to a single bank and performs softmax using the near-bank units in the PIM dies. As a result, SoftmaxPIM achieves $1.19\text{--}1.24\times$ and $1.39\text{--}1.45\times$ speedup over AttAcc with 4 and 8 PIM dies, respectively.

C. Power & Area

We extrapolate the area to the default PIM execution configuration with four PIM-enabled dies per HBM stack. The total softmax-unit area is 0.057mm^2 , corresponding to 6.06% of the 0.94mm^2 PU area in HBM-PIM [1]. This indicates that SoftmaxPIM incurs only modest area overhead. The softmax unit also consumes only 54.01% of the power of a SIMD FP16 adder implemented with the same technology node, remaining within the PIM execution-unit power budget.

D. Model Accuracy

Table II exhibits the conversion-induced numerical loss and the end-to-end WikiText-103 inference accuracy. The Softmax-PIM results reflect all hardware-induced accuracy degradation factors, such as data format conversions and bit truncation for LUT index generation. Following the methodology of [3], we

evaluate the provided nanoGPT-based model trained for 30,000 iterations. The softmax baseline achieves 43.67 perplexity, while ConSmax achieves 46.70, corresponding to only about a 6.93% increase and less than a 1 percentage-point drop in token accuracy. Applying hardware-oriented numerical conversion to the ConSmax checkpoint introduces negligible additional degradation: the ConSmax paper-style INT8/LUT conversion achieves 46.70 perplexity, and SoftmaxPIM achieves 46.72 perplexity. For numerical conversion accuracy, using 1,000 real nanoGPT attention rows of length 256 with FP16 max-shifted scores as reference, ConSmax incurs an exponential mean absolute error (MAE) of 1.88×10^{-2} and a softmax MAE of 1.97×10^{-4} , whereas SoftmaxPIM reduces these errors to 6.29×10^{-5} and 3.66×10^{-7} , respectively.

V. CONCLUSION

In this paper, we present SoftmaxPIM, which integrates a lightweight near-bank softmax unit with GeMV execution to support the GeMV-softmax stage of attention within the memory stack. By processing softmax directly from the bank-level GeMV outputs, SoftmaxPIM reduces data movement and post-GeMV serialization overhead. Our evaluation shows that SoftmaxPIM alleviates softmax bottlenecks to improve scalability and efficiency compared to CPU and GPU baselines, with minimal area and power overhead.

ACKNOWLEDGMENTS

The EDA tool was supported by the IC Design Education Center (IDEC), Korea.

REFERENCES

- [1] S. Lee et al., "Hardware architecture and software stack for PIM based on commercial dram technology: Industrial product," in *Proc. ACM/IEEE 48th Annu. Int. Symp. Comput. Archit.*, 2021, pp. 43–56.
- [2] K. Hong et al., "FlashDecoding++: Faster large language model inference with Asynchronization, Flat GEMM Optimization, and Heuristics," in *Proc. Mach. Learn. Syst.*, 2024, pp. 148–161.
- [3] S. Liu et al., "ConSmax: Hardware-friendly alternative softmax with learnable parameters," in *Proc. 43rd IEEE/ACM Int. Conf. Comput.-Aided Des.*, 2025, pp. 1–9.
- [4] S. Li, Z. Yang, D. Reddy, A. Srivastava, and B. Jacob, "DRAMsim3: A cycle-accurate, thermal-capable dram simulator," *IEEE Comput. Archit. Lett.*, vol. 19, no. 2, pp. 106–109, Jul.–Dec. 2020.
- [5] A. Grattafiori et al., "The llama 3 herd of models," 2024.
- [6] P. Hübner, A. Hu, I. Peng, and S. Markidis, "Apple vs. oranges: Evaluating the apple silicon m-series SoCs for HPC performance and efficiency," in *Proc. IEEE Int. Parallel Distrib. Process. Symp. Workshops*, 2025, pp. 45–54.
- [7] NVIDIA, "NVIDIA ampere GA102 GPU architecture," Rep. JESD235A, 2020. [Online]. Available: <https://nvidia.com/content/PDF/nvidia-ampere-ga-102-gpu-architecture-whitepaper-v2.pdf>
- [8] JEDEC Solid State Technology Association, "JESD235A: High bandwidth memory (HBM) dram," JEDEC, Arlington, VA, USA, Tech. Rep., 2016.
- [9] J. Park et al., "AttAcc! Unleashing the power of PIM for batched transformer-based generative model inference," in *Proc. 29th ACM Int. Conf. Architectural Support Program. Languages Operating Syst.*, 2024, pp. 103–119.
- [10] S. Sarangi and B. Baas, "DeepScaleTool: A tool for the accurate estimation of technology scaling in the deep-submicron era," in *Proc. IEEE Int. Symp. Circuits Syst.*, 2021, pp. 1–5.
- [11] F. Devaux, "The true processing in memory accelerator," in *Proc. IEEE Hot Chips 31 Symp.*, 2019, pp. 1–24.